

5. Abstração I: Encapsulamento

Pratt & Zelkowitz

1

Recursos para programar grandes sistemas

- 5.1 Encapsulamento por subprograma.
- 5.2 Definição de tipos.

Subprograma como Operação Abstrata

Implementação de subprograma

2

- Subprograma representa parte da VM construída pelo programador:
 - ▣ é implementado usando a estrutura de dados e as operações providas pela LP.
- Implementação
 - Encapsulado {
 - cabeçalho → assinatura
 - declarações → locais
 - comandos → definem a ação
 - ▣ uso: apenas via chamada ao subprograma.

Subprograma como OA

Implementação de subprograma

3

- Subprogramas locais:
 - ▣ são subprogramas definidos aninhados, dentro de outros subprogramas (Pascal e Ada).
 - não são acessíveis externamente; estão encapsulados
- Chamada de Subprogramas
 - ▣ invocação checa os tipos dos argumentos,
 - ▣ checagem é estática ou dinâmica,
 - ▣ pode ocorrer coerção automática dos argumentos.

Encapsulamento p/ Subprograma

Definição e invocação de subprograma

4

□ Definição

- é a forma (estática) escrita do subprograma,
- é a única informação disponível em tempo de tradução (só o tipo das variáveis é conhecido).

□ Ativação

- existe somente durante a execução,
- l-value e r-value podem ser acessados, mas os tipos dos OD podem não estar disponíveis,
- usa a definição como gabarito, de modo similar ao tipo de dados (para definir tamanho e RA)

Encapsulamento p/ Subprograma

Definição e invocação de subprograma

5

- A definição de subprograma e sua área de ativação: analogia com tipo e OD daquele tipo.
- No programa em execução:
 - ▣ cria-se uma área de ativação a cada chamada,
 - ▣ quando o subprograma completa sua execução, a área de ativação é destruída,
 - ▣ de uma única *definição* de subprograma, muitas áreas de ativações podem ser criadas,

Encapsulamento p/ Subprograma

Definição e invocação de subprograma

6

- A tradução da definição do subprograma permite dimensionar as RA para os OD e código executável, em tempo de tradução.
 - ▣ esse resultado é usado como um gabarito para as ativações do subprograma.
 - ▣ esse gabarito é dividido em duas partes:
 - segmento de código
 - registro de ativação

Encapsulamento p/ Subprograma

Definição e invocação de subprograma

7

- Segmento de código:
 - ▣ parte estática, *invariante* durante a execução: consiste de constantes e do código executável.
 - ▣ cópia única é *cotizada* por todas as ativações.
- Registro de Ativação
 - ▣ parte dinâmica que contém as informações que variam durante a execução: resultado das funções, parâmetros, variáveis locais e estruturas para housekeeping (dados não locais, temporários, pontos de retorno, etc.).

Encapsulamento p/ Subprograma

Definição e invocação de subprograma

8

```
float fn (float x, int y)
{ const int vi = 2;
  #define vf 10
  float m[vf]; int n;
  ....
  n = vi;
  if (n < vi) { ... };
  return (20*x+m[n]);}
```

1. **fn: float x int $\rightarrow$ float**
Dá as informações p/ RA dos parâmetros e do resultado.
2. **Declarações de m e n induz a RA deles.**
3. **RA para literais e constantes:**
vi é constante, vf é definida como constante 10 (macro); 10 e 20 são literais.
4. **RA para armazenar o código executável criado a partir dos comandos.**

Prólogo para criar o registro de ativação
Código executável
Epílogo p/ eliminar o registro de ativação
20
10
2

Segmento de código para a função fn

Ponto de retorno e outros dados do sistemas
Dados resultantes de fn
x: parâmetro
y: parâmetro
m: ODE local
n: OD local
.....

**Registro de ativação
p/ fn (gabarito)
Um por chamada**

Definição e invocação de subprograma

ODE registro de ativação

10

- Tamanho: determinado durante a tradução.
- Seleção dos componentes:
 - ▣ como em um registro: $\alpha + \text{deslocamento}$.
 - ▣ sua RA é similar a um OD do tipo registro.
- Criação dinâmica:
 - ▣ um registro a cada chamada do subprograma, sendo destruído ao término da rotina.
- Recursão
 - ▣ a cada chamada recursiva é criado um novo registro de ativação.

Definição e invocação de subprograma

Subprograma genérico

11

- Único nome, denominando subprogramas distintos com diferentes assinaturas:
 - ⇒ sobrecarregado,
 - ▣ individualização do subprograma é feita pelo tradutor:
 - com base em seus argumentos.
 - ▣ traz vantagens sem complicar a implementação
- É uma propriedade central em LP tipo ML (tipos polimórficos) ou Prolog.

Subprograma Genérico

Exemplo em ML

12

```
fun size [ ]          = 0
    | size (x::xs) = 1 + size xs;
```

```
> val size = fn: 'a_list --> int
size ["a","perigo", "João", 2,3,4,5]
> 7: int
```

Subprograma Genérico

Exemplo em ADA

13

```
procedure entra (aluno: in integer;  
                 turma: in out Tturma) is  
    begin  
        ....  
    end;  
  
procedure entra(turma: in Tturma;  
                tab: in out TlistaTurma) is  
    begin  
        ...  
    end;
```

Subprograma Genérico

Exemplo em Fortran 90

14

```
INTERFACE ENTRA
```

```
  SUBROUTINE ENTRA1 (ALUNO,TURMA)
```

```
    INTEGER :: ALUNO
```

```
    TTURMA  :: TURMA
```

```
    ....
```

```
  END SUBROUTINE ENTRA1
```

```
  SUBROUTINE ENTRA2 (TURMA,TAB)
```

```
    TTURMA  :: TURMA
```

```
    TLISTATURMA :: TAB
```

```
    ...
```

```
  END SUBROUTINE ENTRA2
```

```
END INTERFACE ENTRA
```

Abstração I: Encapsulamento

Definição de tipos

15

- Definir tipos abstratos de dados requer:
 - a) definir um novo tipo ou classe de dados.
 - C, Pascal e Ada permitem definição de tipos
nome do tipo $\Rightarrow$ nome da classe de objetos de dados
type racional = **record**
 numerador, denominador: **integer**;
end;
var A, B, C: racional;
 - b) mecanismos para definir as operações sobre o novo TD.
 - especialidade de linguagens orientadas a objetos.

Definição de Tipos

Exemplo em C

16

```
struct racional {  
    int numerador, denominador;  
}
```

```
struct racional A, B, C;
```

- ▣ uso de struct para definir variáveis viola o princípio:
 - tudo que o programador vê é o nome do tipo e a lista de operações para manipulá-lo.

```
typedef struct {int numerador, denominador;} racional;  
racional A, B, C;
```

- ▣ sintaxe semelhante ao Pascal: não viola a regra de TAD
- ▣ typedef é similar a uma macro:
 - novos tipos são criados em C pela declaração struct.

Definição de Tipos

17

- Uso: gabarito para definir OD na execução:
 - ▣ permite separar a definição da estrutura do OD da definição dos pontos nos quais OD desse tipo serão criados na execução do programa.
- Constitui uma forma de encapsular dados e esconder informação:
 - ▣ se subprograma cria OD usando o nome do tipo e não acessa diretamente os seus componentes internos $\Rightarrow$ a definição do tipo pode mudar sem ser necessário alterar o programa.

Definição de Tipos

Implementação

18

- Similar a declarações, *definição de tipo* é útil apenas na tradução para determinar a RA do OD, checagem de tipo e gerência de armazenamento:
 - ▣ durante a compilação, definições de tipo são inseridas em uma tabela específica e usadas quando OD deste tipo são declarados.
 - ▣ gerado o código objeto, não são necessárias:
 - em LP compiladas os objetos de dados não tem etiqueta de tipo.

Definição de Tipos

Equivalência de Tipos

19

- Checagem de tipo (estática ou dinâmica)
 - ▣ comparação entre os tipos dos argumentos *atuais* e esperados (*formais*) de uma operação.
 - ▣ um erro ou coerção ocorre, se os tipos não são os mesmos.
- Questões em equivalência de tipos:
 - ▣ quando dois OD são do *mesmo tipo*?
 - questão tratada em tipos de dados.
 - ▣ quando dois OD do mesmo tipo são *iguais*?
 - questão semântica relativa ao r-value de um OD.

Definição de Tipos

Equivalência de Tipos

20

```
program Pxy ;  
  type T1 = array[1..10] of  
    real;  
        T2 = array[1..10] of  
    real;  
  var x,z: T1; y: T2;  
  procedure Sub(A:T1);  
    begin  
      ...  
    end;  
begin    {ADA, C++}  
  x:=y;  {erro: não Pascal}  
  Sub(y);{erro tipo Pascal}  
end ;
```

- a) x, y, z e A tem o mesmo tipo?
- b) É válido fazer
 x:=y? e Sub(y)?
- c) Há 2 enfoques básicos:
 - equivalência de nomes.
 - equivalência estrutural.
- d) Equivalência de nomes:
 TD são equivalentes se têm o
 mesmo nome.
 - usado em ADA, C++ e em
passagem de parâmetros em
Pascal.

Equivalência de Tipos

Equivalência de nomes

21

□ Desvantagens:

- todo OD usado em atribuição *precisa* ter um nome de tipo. Não pode haver tipos anônimos.

var w: array [1..10] of real;

- não é argumento válido de subprograma.
- TD de argumento transmitido em uma cadeia de subprogramas não pode ser novamente definido em cada subprograma:
 - única definição global de tipo deve ser usada.
 - definição de classe em C++, package em ADA e arquivos de inclusão “.h” em C garantem isto.

Equivalência de Tipos

Equivalência Estrutural

22

- Dois tipos de dados são equivalentes se definem OD com os *mesmos* componentes internos:
 - ▣ isto é, ambos tem a mesma RA, em tempo de execução e mesma forma para selecionar componentes.
 - ▣ T_1 e T_2 são equivalentes estruturais porque têm a mesma RA.

Equivalência de Tipos

Questões sobre equivalência estrutural

23

- Registros são equivalentes se componentes:
 - ▣ estão na mesma ordem e têm os mesmos tipos?
 - ▣ mesma ordem, têm os mesmos tipos e nomes?
 - ▣ tendo mesmos tipos e nomes, precisam estar na mesma ordem?
- Arranjos com mesmo n° de componentes de tipos iguais: subscritos precisam ser iguais?
- Literais em dois tipos de enumerações precisam ser os mesmos e estarem na mesma ordem?

Equivalência de Tipos

Questões sobre equivalência estrutural

24

- Equivalência equivocada:

```
type Tmetro = integer;
```

```
    Tlitro = integer;
```

```
var dis: Tmetro;  - distância em metros
```

```
    vol: Tlitro;   - volume em litros
```

```
X:= dis + vol ;   - Equivalência estrutural: OK.  
                  - Equivalência de nome: erro.
```

- O programador declarou tipos diferentes e considerá-los iguais é um erro semântico.

- Compilação cara (frequência da)
verificação de equivalência entre OD complexos

Equivalência de Tipos

Projeto de linguagens

25

- Em LP como Pascal e Ada equivalência de tipos tem um papel central.
- Em LP antigas (Cobol, Fortran, PL/1), não há definição de tipos; usa-se formas de equivalência estrutural.
- C usa equivalência estrutural.
- C++ e Ada usam equivalência de nomes.
- Equivalência é um assunto de pesquisa.

Equivalência de Tipos

Igualdade de OD

26

- Se dois objetos A e B têm o mesmo tipo, quando são iguais? Isto é $A = B$?
- ▣ Resposta é não trivial

struct tpilha {**int** topo; **int** dados[100]} x,y;

struct tconjunto {**int** num; **int** dados[100]} a,b;

1) a, b, x e y são estruturalmente equivalentes.

2) as condições sobre as quais se deseja ter $a=b$ e $x=y$ podem ser muito diferentes.

Equivalência de Tipos

Igualdade de OD

27

- Igualdade de pilha
 - i) $x.topo = y.topo$ // aponta para mesmo índice.
 - ii) $x.dados[k] = y.dados[k]$, para $k=0, x.topo-1$.
- Igualdade de conjuntos
 - i) $a.num = b.num$ // mesmo # de elementos.
 - ii) $a.dados[0], \dots, a.dados[a.num-1]$ é uma permutação $(b.dados[0], \dots, b.dados[b.num-1])$
- Não é fácil formalizar igualdade p/ ODE, com tipos criados pelo usuário:
 - ▣ cabe ao usuário essa tarefa.

Definição de Tipos

Definição de tipos com parâmetros

28

- Tipo parametrizado facilita definir tipos de dados similares, p.ex. arrays de $\neq$ tamanhos

1) exemplo com parâmetros *explícitos* em Ada

type mês (dias: **integer range** 1 .. 31) **is**

record

nomes: **string**(1 .. 3);

vendas: **array**(1 .. dias) **of float**

end record;

fev: mês(28); m30: mês(30); m31: mês(31);

Número de dias faz parte da declaração do mês.

Definição de Tipos

Definição de tipos com parâmetros

29

2. Exemplo sem parâmetros *explícitos* em Ada

a) type matriz is array (integer range <>, integer range <>) of float;

- tabela: matriz(1..10,1..20);

b) novalista (1..n) of tipox;

- Cada vez que o subprograma contendo esta declaração for ativado tem-se um arranjo novalista de comprimento n, variável externa ao subprograma.

□ Implementação

- definição funciona como um molde (template). O parâmetro deve ser definido antes de usado.